AI Invoice Assistant

Architecture and technical exploration

Version 1.4 - 11 October 2026

Changes since v1.3

- Storage: SQLite (`data/invoices.db`) instead of JSON files.
- One chat for the website (`/chat`): a router bot picks the change bot or the assistant; the per-area chat endpoints remain.
- Questions: an assistant with tools replaces the question bot; turnover questions use forms (choosers).
- No data block in the prompt: the bots write names, Python looks the records up; Python's errors go straight to the user (no retry by the bot).
- Endpoint names follow the code (section 1.4); a website is served by the API.
- Risk countermeasures that are not built in v1 are marked as such (section 4).

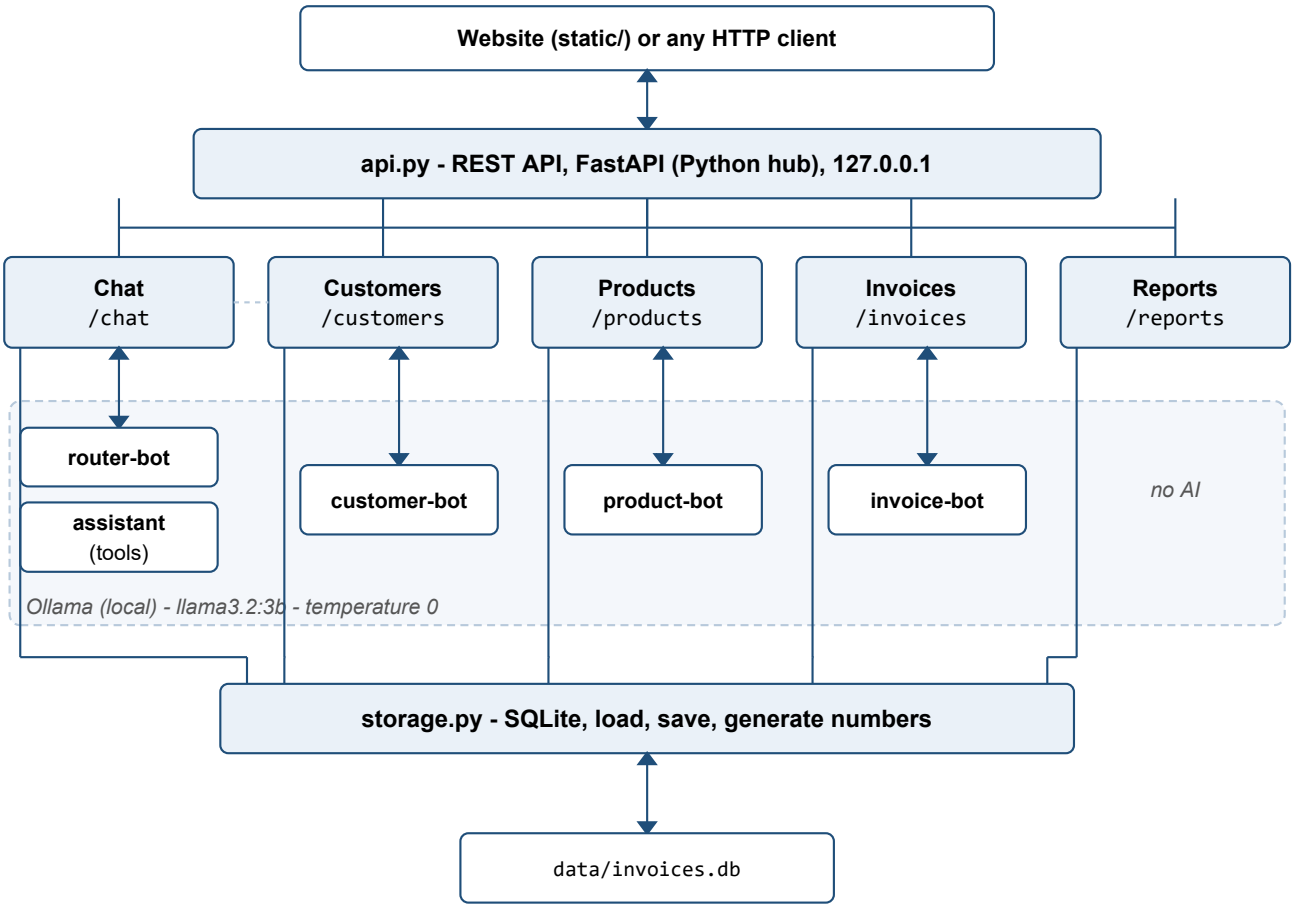
1. Service design

1.1 Options considered

Question	Options considered	Chosen	Why
How many bots?	One bot for everything / one bot per area	One change bot per area, plus a router bot and an assistant	A 3B model follows short, focused prompts more reliably; a change bot that also answers questions mixed the two up ("X has moved" read as a question)
How do the bots coordinate?	Bots call each other / Python as the hub	Python hub	The router bot only names a bot; Python passes the message on. A 3B model can't coordinate reliably
Chat entry	One chat per area / one chat for everything	One chat (`/chat`) with a router; per-area chats remain	The user doesn't have to pick the area; an open task (a question or an error) stays with the same bot
Data to the bot	All data in every message (data block) / names only, Python looks up	Names only	With the data block the bot copied example names and claimed records didn't exist; it also doesn't scale
Questions	Question bot with text commands / bot with read tools	Assistant with tools (Ollama tool calling)	Read tools are answered by Python; write tools only return a proposal; forms supply periods and records
Who calculates?	The AI / Python	Python	Small models make arithmetic mistakes; turnover results carry a Total row
Who confirms?	The AI asks yes/no / the API stores a pending change / the client sends the checked proposal to a CRUD endpoint	Client, after the user's yes	The bot skipped its own confirmation in tests; nothing has to be stored before the user agrees
Separation	Chat and saving in one flow / chat endpoints that only propose + plain CRUD endpoints	Chat proposes, CRUD saves	The CRUD part works and can be tested without AI, and other programs can use it
Storage	JSON files / SQLite / server database	SQLite	One file, built into Python, no server; a save is one transaction (all or nothing). JSON files were used until episode 6
Interface	Terminal / REST API / web GUI	REST API plus a website served by the API	Any client can use the API; the website is one such client

Question	Options considered	Chosen	Why
API framework	FastAPI / Flask	FastAPI	Validates requests automatically and generates interactive documentation (`/docs`)
Conversation state	Client sends the whole history each time / server keeps the conversation	Server, by `conversation_id`	The client stays simple and can't change what the bot said before
Access	Open on the network / only on this computer	Only `127.0.0.1`, no login in v1	No authentication is needed while the API can't be reached from outside
Calling Ollama	`ollama run` on the command line / REST API / `ollama` Python library	REST API with Python's `urllib`	`ollama run` packs everything into one text message (worse results in tests); `urllib` needs no extra library

1.2 Architecture



The router bot only names a bot; Python passes the message on. The bots never talk to each other and never touch the database. Every exchange passes through Python.

1.3 Components

Component	Role
`app/api.py`	FastAPI application: CRUD endpoints, reports, chat endpoints (conversations by conversation_id); serves the website
`app/router.py`	The single chat: asks the router bot which bot a message belongs to; keeps an open task with the same bot; tells the website which chooser to show
`app/assistant.py`	Questions: llama3.2:3b with tools. Read tools are answered by Python at once; write tools are checked and returned as a proposal; UI tools open the turnover forms. Answers in three parts (Answer, Input, Analysis) plus a line listing the data Python read
`app/bot.py`	Calls Ollama, reads the bot's command, Python guards (invented names, numbers, years and dates; placeholders; duplicate keys), and the conversation of the change bots
`app/customers.py`, `products.py`, `invoices.py`	Per entity: the checks (used by the chat **and** again by CRUD), lookup by name, the proposal with a summary, and the save functions used by CRUD
`app/reports.py`	R1, R2, R3: turnover (paid invoices, by due date) from the given dates or year and filters. No AI.
`app/storage.py`	SQLite (`data/invoices.db`): loads and saves, generates numbers (CUS-001, PRD-001, INV-001)
`static/`	The website: one chat, Yes/No for proposals, choosers (dates, customers, products, invoice lines, turnover forms), tables
`modelfiles/`	System prompts of the router bot and the customer, product and invoice bots

1.4 Endpoints

Endpoint	Purpose	AI
`POST /chat`	The single chat used by the website: the router sends the message to a change bot or to the assistant. Returns a reply (with the name of the bot, and which chooser to show) or a checked proposal. **Never saves anything.**	Yes

`POST /customers/chat`
`POST /products/chat`
`POST /invoices/chat`
`POST /assistant/chat`
	Chat with one bot directly. Body: `conversation_id` (optional), `message`. Never saves.	Yes

`POST /customers`, `PUT /customers/{id}`, `DELETE /customers/{id}` **same for** `/products` **and** `/invoices`	Create, update, delete. Checks the data again, then saves and assigns the number. The client calls these after the user said yes to the summary.	No
`GET /customers`, `GET /customers/{id}` **same for** `/products` **and** `/invoices`	Retrieve all records or one record (an invoice with its lines and total)	No
`GET /reports/customers`	R1. Parameters: `start`, `end` (DD/MM/YYYY)	No
`GET /reports/products`	R2. Parameters: `start`, `end`	No
`GET /reports/months`	R3. Parameters: `year`, optional `customer_id`, optional `product_id`	No

1.5 Flow of a change (example: "delete Wilfried")

1. The website sends `POST /chat` with the message "delete Wilfried". The router bot answers "customer", so Python passes the message to the customer bot with the earlier messages of this task.
2. The bot replies with a question, or with one command such as `DELETE_CUSTOMER {"name": "Wilfried"}`. It never sees the data.

3. Python guards the command: a name, invoice number, year or date the user never wrote leads to a question ("Which customer do you mean?"); placeholders are emptied; broken JSON or duplicate keys give "Sorry, I didn't understand that".
4. Python checks the command: it looks the name up (none / several / one), checks the values and whether the customer is used in an invoice. An error goes straight to the user, and the next message stays with the same bot.
5. If it's valid, the API returns the `proposal` and the `summary` "Delete customer CUS-002: Wilfried Becker | address". Nothing is saved.
6. The website shows the summary with Yes / No. On "No" nothing happens.
7. On "Yes" the website calls `DELETE /customers/CUS-002`. Python checks again (the data may have changed in the meantime), deletes and returns the result.

1.6 Flow of a question (example: "turnover per customer")

1. The router sends the message to the assistant. Without a period it calls the UI tool `ask_turnover_period`: the website shows a form (from, to, optional customer and product).
2. If the assistant instead calls `get_turnover_by_customer` with a period the user never wrote, Python shows the form anyway.
3. The form's answer comes back as a message (with ids in brackets). The assistant calls the read tool; Python calculates the turnover, filters the customer or product and adds a Total row.
4. The assistant answers (Answer, Input, Analysis); Python adds the line "Data read by Python: ..." so the user can compare the answer with the data.

1.7 Division of responsibility

AI (bots)	Python (API)	Client (website)
Understanding natural language	Guards and checks (in the chat and again before saving)	Shows the summary, Yes / No
Picking the bot (router)	Looking records up by name; numbers and codes	Shows the choosers and forms
Asking for missing fields	All calculations (totals, turnover)	On yes: calls the CRUD endpoint
Choosing tools (assistant)	Forcing the form when a period is invented or missing	
Phrasing answers	Conversation state and saving	

2. Model selection

2.1 Requirements for the model

- It runs **locally and offline**, so no customer data leaves the computer.
- It runs on an ordinary office PC with no dedicated GPU, with acceptable response times.
- It follows instructions reliably: one question at a time, and an exact command format.
- It produces valid JSON in its commands.
- It understands names, dates and quantities in short user messages.

2.2 Candidates considered

Model	Size on disk	Strengths	Weaknesses	Decision
`llama3.2:1b`	1.3 GB	Very fast, small	Weak at following multi-step instructions and keeping a format	Rejected: too unreliable
`llama3.2:3b`	2.0 GB	Good balance of speed and instruction following; designed for on-device use	Makes mistakes in arithmetic and long conversations	**Chosen**
`qwen2.5:7b`	4.7 GB	Stronger reasoning and JSON output	More than twice as large, noticeably slower without a GPU	Rejected; kept as a fallback
Cloud models (GPT, Claude, ...)	-	Most capable	Data leaves the company, needs internet, costs per use	Rejected: offline and privacy requirement

The task description specifies `llama3.2:3b`. This comparison confirms it as a sensible choice, as long as the design makes up for its weaknesses: Python does the calculations, the validation and the confirmation (see section 1).

2.3 Parameters

Parameter	Value	Reason
`temperature`	0	The same input always gives the same output, so tests can be reproduced and the format stays stable
`num_ctx` **(context window)**	Ollama's default	The change bots get no data block, only the conversation. The assistant gets tool results (lists of all records), which grow with the data (see RK8)

2.4 Customisation method

Option	Decision
Fine-tuning the model	Rejected: needs training data, time and hardware
System prompt plus data in each message	Replaced: the data block was dropped (section 1.1)
System prompt only: in a Modelfile (`modelfiles/`) for the router and change bots, in `assistant.py` for the assistant	**Chosen**: quick to change, each bot has its own small prompt; examples use made-up names

3. Prototypes

The prototypes were built step by step before the application. Each one answered one question, and its result decided the next design step. Sections 3.1 to 3.4 record them as they were; section 3.5 records what changed during implementation.

3.1 Overview

#	Prototype	Question	Result	Decision
P1	One bot for everything	Can one system prompt handle all CRUD flows and the questions?	Designed but not built: one prompt would have to cover 12+ different flows	One bot per area
P2	Customer bot v1, called with `ollama run`	Can the bot run a create/delete conversation including the yes/no confirmation?	Partly	Always send the data; never let the bot write a data block itself
P3	Same bot called through the chat API, with 2 data formats	Which message format works best?	Format B found the right customer, but skipped the confirmation	Use format B; Python asks for the confirmation
P4	Customer bot v2, with no confirmation in the prompt	Does the bot produce correct commands over several turns?	Finds the right customer, but sloppy in the flow	Python checks every command
P5	Python validation layer	Can Python catch every bot mistake seen in P2-P4?	21 out of 21 test cases passed	Keep this design

3.2 Observed bot behaviour (P2-P4)

Test	What the bot did	Problem
"add a new customer", sent without data	Asked for the name, but **invented** a data block with two fake customers (John Doe, Jane Doe)	The bot invents data
Same, with an empty data block	Asked the right question, but **repeated the data block** in its reply	Internal data visible to the user
"delete Maria", using `ollama run`	**Invented** a customer "John Doe, CUS-002" instead of finding Maria	Wrong record
"delete Maria", format A (plain text)	Asked for confirmation without showing which customer	Unclear confirmation
"delete Maria", format B (tags + JSON)	Correct ID `CUS-001`, but **deleted with no confirmation**	Dangerous
"delete Peter" (unknown customer)	Asked again, but didn't say that Peter doesn't exist	Unclear message
"change Maria's email"	Sent the command **too early**, with `"value": "..."`	Incomplete command
Creating a customer over 5 turns	Accepted an invalid email at first and only objected one question late; the final command was correct	Validation by the bot is unreliable

3.3 Lessons learned

1. The bot is good at **understanding** the user.
2. The bot is unreliable at **process rules**: confirmation, the order of questions, and validation.
3. So every rule that has to hold is enforced in **Python**, not in the prompt: validation, the yes/no confirmation, number generation and calculations.
4. (Replaced in v1.4) The data was sent as tagged JSON (format B); see 3.5.

3.4 Test results of the validation layer (P5)

Function	Cases tested	Result
Reading the command (`parse_command`)	Valid command, placeholder, normal question, broken JSON, echoed data block	5 out of 5 OK
Validation (`validate`)	Valid create/update/delete, missing value, invalid value, changing the number, unknown number, customer used in an invoice	9 out of 9 OK
Confirmation text (`describe`)	Create, update, delete	3 out of 3 OK
Saving (`apply`)	Create x2, update, delete, numbers not reused, spaces removed, extra field dropped	4 out of 4 OK

3.5 Findings during implementation

Finding	Decision
With the data block, the bot copied example names and claimed existing records didn't exist	No data block: the bot writes names, Python looks them up
When a message names no record, the bot fills in example names or numbers ("Lisa Kraus", "INV-031")	Python guards: `invented_target`, `invented_period`, `clean_command`, `unique_keys`
A judge or coach bot checking the answers made results worse	Dropped
A change bot that also answers questions confused the two	Questions go to a separate assistant
The assistant ignored rules in the prompt and invented periods ("2031") or ids ("FEL-001")	Forms (choosers) supply periods and records with ids; Python forces the form and checks every id
The bot added up totals wrongly	Python returns a Total row; the bot copies it

4. Risk analysis

4.1 Scale

- **Likelihood (L):** Low / Medium / High, meaning how likely the risk is to happen without any countermeasure.
- **Impact (I):** Low / Medium / High, meaning how much damage it does to the data or to the user.
- **Residual risk:** what's left after the countermeasure. "Not in v1" means the countermeasure isn't built (only the core functionality was implemented).

4.2 AI behaviour

ID	Risk	Evidence	L	I	Countermeasure	Residual
RK1	The bot invents records or numbers	Seen in P2 and in the danger tests	H	H	Python checks that every name, invoice number, year and date in a command was written by the user; ids from tools are checked; numbers are generated only by Python	Low
RK2	The bot saves or deletes without confirmation	Seen in P3	H	H	The chat never saves: it only returns a proposal. Saving needs a separate CRUD call by the client after the user's yes	Low
RK3	Incomplete or invalid commands (`"..."`, invalid values)	Seen in P4	H	M	Python validation; the error goes to the user and the task stays with the same bot; placeholders are emptied, broken JSON is refused	Low
RK4	Wrong numbers in answers to questions	Seen: invented total, wrong row picked	H	H	Python calculates, filters and adds the Total; forms supply the period; the line "Data read by Python" shows what the bot got	Medium
RK5	The bot picks the wrong record (similar names, a name that is part of a longer one)	Seen: "Paul Müller" matched Paul Müller-Lang	M	H	The summary shows the full record before anything changes; choosers offer exact records	Low
RK6	Internal data shown to the user	Seen in P2 and P4	H	L	No data block is sent to the change bots anymore	Low
RK7	Text inside the data is read by the bot as an instruction	Possible	L	M	Change bots get no data; the assistant gets tool results as JSON; every change is validated and confirmed	Low
RK23	The router sends a message to the wrong bot	Seen in tests	M	M	Each answer names the bot; an open task stays with its bot; a change bot can't read data, the assistant can't save	Low

4.3 Technical

ID	Risk	L	I	Countermeasure	Residual
RK8	The assistant's tool results no longer fit in the context window as the data grows	M	H	Not in v1 (`num_ctx` default; the tools return whole lists)	Medium
RK9	Slow answers on a PC without a GPU	M	M	Small model, short prompts; target of 10 seconds (AC-G7)	Medium
RK10	Ollama not running, or the model missing	M	M	Not in v1: the API answers with a server error	Medium
RK11	The database is damaged (crash while writing)	L	H	SQLite writes each save in one transaction (all or nothing)	Low
RK12	Data loss (disk failure, deletion)	L	H	Outside v1; recommend backing up the `data/` folder	Medium
RK13	A model update changes the bot's behaviour	L	M	Model named in the Modelfiles; re-run the test sets after any update	Low
RK18	Unauthorised access to the API	M	H	The API listens only on `127.0.0.1`; no login needed in v1. A login is required before opening it to the network	Low
RK19	Two requests change the same data at the same time (one change gets lost)	M	H	Not in v1 (no lock around load-change-save)	Medium
RK20	The client gives up (HTTP timeout) while the bot is still answering	M	M	Bot target of 10 seconds (AC-G7); clients use a timeout of at least 30 seconds	Low
RK21	Conversations are lost on restart, or pile up in memory	M	L	Not in v1 (no expiry); the client simply starts a new conversation	Low
RK22	A confirmed proposal arrives after the data has changed (e.g. the customer was already deleted)	L	M	The CRUD endpoints run the same checks again before saving	Low

4.4 Functional

ID	Risk	L	I	Countermeasure	Residual
RK14	Turnover is misunderstood (unpaid invoices counted, wrong date used)	M	M	Exact definition in the requirements; Python counts paid invoices by due date; cross-checks AC-R3 and AC-R4	Low
RK15	Date formats get confused (DD/MM vs MM/DD)	M	M	Python accepts only DD/MM/YYYY; the date choosers avoid typing. Dates written out in the confirmation: not in v1	Low
RK16	Deleting a customer or product that is still used in invoices	M	H	Python refuses the deletion (AC-C4, AC-P3)	Low
RK17	`llama3.2:3b` turns out not to be good enough overall	M	H	Fallback to `qwen2.5:7b`: only the model line in the Modelfiles changes	Medium

4.5 Summary

The risks observed with the bots are reduced to **Low** by rules enforced in Python, except the numbers in answers to questions. These risks remain at **Medium**:

- RK4: numbers in answers to questions,
- RK8: context window size for the assistant,
- RK9: speed,
- RK10, RK19: not built in v1 (Ollama error message, lock for parallel changes),
- RK17: model quality.

RK12 (data loss) is also still Medium, but it's outside v1.